

# Analysis: Checksum

Suppose we know a subset  $S$  of elements of the matrix  $\mathbf{A}$  because they are either given in the input or revealed by Edsger. Can we restore the remaining elements? If there is only one unknown element in some row or column, we can recover its true value from the known elements in that row or column and the respective checksum. We can apply such an element recovery step until all elements are known or we end up with a matrix where each row and column has at least two unknown elements (aside from fully reconstructed rows and columns). It will be shown later that the whole matrix can be restored using this simple strategy or the subset  $S$  does not provide enough information to do so.

## Test Set 1

For the small test set, we can examine every potential subset  $S$  of matrix elements and check if  $S$  is sufficient to restore the whole matrix. Among all sufficient subsets  $S$ , we pick the smallest in terms of hours as our answer.

Given a subset  $S$  of known elements, the above matrix recovery strategy can be implemented in  $O(N^2)$  time. For example, we can use a BFS-like algorithm iterating over a queue of elements that are the only unknown elements in their rows or columns and calculating their true values one by one. For the sake of efficiency, we also need to maintain the XOR values of currently known elements for each row and column as well as the number of unknown elements per each row and column. As soon as the number of unknown elements for a row or column becomes 1, we add that element to the queue.

There are  $O(2^{N^2})$  subsets of elements. Consequently, the overall time complexity of this brute-force algorithm is  $O(2^{N^2} \times N^2)$ .

## Test Set 2

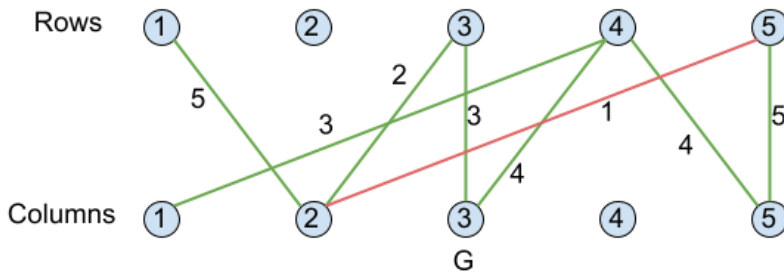
Let us look at our problem from graphs perspective. Namely, we construct a weighted bipartite graph  $G$ , where the rows and columns of the matrix  $\mathbf{A}$  are represented by nodes in  $G$ , and there is an edge of weight  $\mathbf{B}_{i,j}$  between  $i$ -th row and  $j$ -th column if and only if  $\mathbf{A}_{i,j} = -1$ . Please see the below example to make the construction more clear.

|   | 1  | 2  | 3  | 4 | 5  |
|---|----|----|----|---|----|
| 1 | 0  | -1 | 1  | 1 | 1  |
| 2 | 1  | 0  | 0  | 1 | 0  |
| 3 | 1  | -1 | -1 | 0 | 0  |
| 4 | -1 | 0  | -1 | 1 | -1 |
| 5 | 0  | -1 | 0  | 1 | -1 |

**A**

|   | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 |   | 5 |   |   |   |
| 2 |   |   |   |   |   |
| 3 |   | 2 | 3 |   |   |
| 4 | 3 |   | 4 |   | 4 |
| 5 |   | 1 |   |   | 5 |

**B**



An isolated node represents a row or column with all its elements known. We can safely disregard such nodes. A leaf node with exactly one incident edge represents a row or column with precisely one unknown element. Note that the process outlined in the first paragraph corresponds to repeated removal of leaves from the graph. If we end up with an empty graph in this way, it means that the original graph must have been a forest without any cycles and we can recover the full matrix without spending any time.

So what if the graph  $G$  does contain a cycle? Given any assignment of binary values to the elements of the matrix, we can flip the values of elements corresponding to the edges of a cycle, and this operation would not change the XOR checksum of any row or column. Consequently, we cannot tell the true value of elements on a cycle unless we reveal at least one of them, and effectively break the cycle by removing the edge from the graph and paying a delicious price. In other words, in order to be able to recover the whole matrix, it is necessary to break all cycles by revealing and removing some edges. It is also sufficient — once all cycles have been broken, what remains is a forest of edges, and the true value of all remaining edges can be determined unambiguously.

Thus we have reduced the original problem to finding a minimum weight subset of edges that breaks all cycles. An intuitive greedy approach involves iterating over the edges in a non-decreasing order of weights and removing the current edge from the graph if it is part of a cycle. An edge is part of a cycle if there is a simple path between its end-nodes other than the edge itself — a condition that can be tested by, say, running a [depth-first search](#) from one of the end-nodes.

In our example above, once we remove the cheapest edge of cost 1 (the red edge), what remains is a tree, so no other edges will be removed.

Since there can be up to  $N^2$  edges, the above steps are repeated  $O(N^2)$  times. One run of the depth-first search costs  $O(N^2)$  time as well, so the overall time complexity of this approach is  $O(N^4)$ .

But what about the correctness of this greedy approach? The proof is very similar to that of [Kruskal's](#) algorithm. Consider the first edge  $e$  that is removed by the algorithm, so it has the smallest weight among all edges on cycles. In particular, suppose that  $e$  is part of a cycle  $C$ . Now, consider any cycle breaking set of edges  $X$  that does not include  $e$ . Since the cycle  $C$

must be broken,  $X$  must contain an edge  $f \neq e$  that is also part of the cycle  $C$ . The set of edges  $Y = X - f + e$  has a total weight no larger than the weight of  $X$  and it is cycle breaking as well. To prove the second claim, assume the contrary that the graph  $G - Y$  contains a cycle  $C'$ , which necessarily includes the edge  $f$  and does not include the edge  $e$ . But then we can combine the paths  $P = C - f$  and  $P' = C' - f$  to form a cycle in  $G - X$ , which contradicts the fact that  $X$  was cycle breaking. It follows that the edge  $e$  is part of some optimal solution and our greedy choice was valid.

### Test Set 3

Of course, the problem of finding a minimum weight cycle breaking edge set is equivalent to the well known problem of finding a [maximum weight spanning forest](#) of  $G$ , except that we would build the complement set of edges to keep rather than the set of edges to remove. In the example above, the edges of the maximum weight spanning forest are rendered green. It would cost Grace one hour (the red edge) to reconstruct the whole matrix.

The graph may potentially have up to  $N^2$  edges, therefore, a [simple implementation](#) of Prim's algorithm without maintaining a priority queue data structure would achieve an  $O(N^2)$  time complexity. Note that because of the high density of the graph, Prim's algorithm is a better choice than Kruskal's algorithm, as the later would need  $O(N^2 \times \log N)$  time for sorting the edges.

It is interesting to note that we never use the actual checksums in the graph construction nor the maximum spanning forest algorithm, therefore, the input values  $\mathbf{R}_i$  and  $\mathbf{C}_i$  can be safely ignored.

# Analysis: K-Goodness String

As per the given definition of operation, Ada can only change the goodness score by one in a single move. Therefore to get a string with the required goodness score in the minimum number of operations Ada can either increase or decrease the goodness score by one in each step. Let us assume there are  $X$  indices ( $\leq N/2$  (1-indexed))  $i$  in the given string  $S$  such that  $S_i \neq S_{N-i+1}$ . We have 3 cases now.

- Case 1:  $X = K$ ,  
In this case, Ada already has the string which has a goodness score of  $K$ . Therefore number of operations required is 0.
- Case 2:  $X > K$ ,  
In this case, Ada can change  $(X - K)$  letters at indices  $i$  ( $1 \leq i \leq N/2$  (1-indexed)) that satisfy  $S_i \neq S_{N-i+1}$  to the letter at  $S_{N-i+1}$ . Then she will have a string with a goodness score of  $K$ . Therefore the minimum number of operations is  $(X - K)$ .
- Case 3:  $X < K$ ,  
In this case, Ada can change  $(K - X)$  letters at indices  $i$  ( $1 \leq i \leq N/2$  (1-indexed)) that satisfy  $S_i = S_{N-i+1}$  to any other uppercase letter other than the one at  $S_{N-i+1}$ . As a result, she will have a string with a goodness score of  $K$ . Therefore the minimum number of operations is  $(K - X)$ .

All the above described operations can be done in  $O(N)$  complexity. Therefore the overall complexity is  $O(N)$ .

### Sample Code (C++)

```
int kGoodnessString(string s, int k) {
    int minOperations = 0, x = 0;
    for(int i = 0; i < s.size() / 2; i++) {
        if(s[i] != s[s.size() - i - 1]) {
            x++;
        }
    }

    if(x == k) {
        minOperations = 0;
    }
    else if(x > k) {
        minOperations = x - k;
    }
    else {
        minOperations = k - x;
    }
    return minOperations;
}
```

# Analysis: L Shaped Plots

## Test set 1

In order to verify that a segment is good, we need to check whether all the cells in that segment contain 1 or not. To check whether all the cells in a segment contain 1 or not, we can calculate prefix sum of the matrix and then check whether sum of cells on this segment is equal to the length of the segment or not. Let  $query(a, b, c, d)$  denote the sum of cells in submatrix with  $(a, b)$  as top left corner and  $(c, d)$  as bottom right corner. We can calculate this sum in  $O(1)$  using the prefix sum matrix. For more details on using prefix sum to calculate sum of cells on a submatrix, please refer [this](#). In order to get the sum of cells on segment from  $(i, j)$  to  $(i, l)$ , we can simply check if  $query(i, j, i, l) = |j - l| + 1$ .

L-shape comprises of two segments which meet at a common point. Except the common point, consider the other end point of one segment as  $(i, j)$  and the end point of the other segment as  $(k, l)$ . Now these end points could meet at either  $(i, l)$  or  $(k, j)$  to form an L-shape. So, if we know the end points of each segment, we can figure out the common point where segments would meet.

For each pair of end points  $(i, j)$  and  $(k, l)$  of segments of L-shape, we already saw that there could be 2 possible meeting points.

- For segments meeting at  $(i, l)$ , check that  $query(i, j, i, l) = |j - l| + 1$  and  $query(k, l, i, l) = |i - k| + 1$ . Besides, either  $|j - l| + 1 = 2 \times (|i - k| + 1)$  or  $|i - k| + 1 = 2 \times (|j - l| + 1)$  should be true. If these conditions are satisfied, increase the answer by 1.
- For segments meeting at  $(k, j)$ , check that  $query(i, j, k, j) = |i - k| + 1$  and  $query(k, l, k, j) = |j - l| + 1$ . Besides, either  $|i - k| + 1 = 2 \times (|j - l| + 1)$  or  $|j - l| + 1 = 2 \times (|i - k| + 1)$  should be true. If these conditions are satisfied, increase the answer by 1.

We can iterate over all possible end points of these two segments L-shape because there are  $O(\mathbf{R}^2 \times \mathbf{C}^2)$  such possible combinations. We can calculate the number of possible L-shapes with these end points in  $O(1)$ . Hence, the overall complexity of the solution is  $O(\mathbf{R}^2 \times \mathbf{C}^2)$ .

## Test set 2

We cannot iterate over all possible end points of the segments for this test set as the solution would time out. If for each cell, we can quickly calculate how many L-shapes are such that both of its segments meet at this cell, we can iterate over each cell of the matrix only once and calculate our answer. We can safely ignore those cells that have value 0 as they cannot be part of any L-shape.

Consider a cell  $(i, j)$ . There can be 4 types of L-shapes that have both of its segments meet this cell.

- Type 1: One of the segments is to top of  $(i, j)$ , and other segment is to the right of  $(i, j)$ .
- Type 2: One of the segments is to top of  $(i, j)$ , and other segment is to the left of  $(i, j)$ .
- Type 3: One of the segments is to bottom of  $(i, j)$ , and other segment is to the right of  $(i, j)$ .
- Type 4: One of the segments is to bottom of  $(i, j)$ , and other segment is to the left of  $(i, j)$ .

Let  $Count(x, y)$  be the number of L-shapes with both its segments meeting at a particular point of which the length of the segment parallel to one axis is  $x$  and the length of the segment parallel to the other axis is  $y$ . Number of L-shapes with longer segment as part of the segment with length  $x$  are  $\min(\frac{x}{2}, y) - 1$ . Similarly, number of L-shapes with longer segment as part of the segment with length  $y$  are  $\min(\frac{y}{2}, x) - 1$ . Hence,  $Count(x, y) = \min(\frac{x}{2}, y) + \min(\frac{y}{2}, x) - 2$ .

If we can calculate number of consecutive cells that have value 1 in each side of  $(i, j)$ , we can calculate number of L-shapes of each type with this cell as the common endpoint of the segments using the  $Count()$  function above. Let  $top(i, j)$  denote the number of consecutive cells that have value 1 including  $(i, j)$  and cells above it. For cells with value 0,  $top(i, j) = 0$ . Formally for cells with value 1,  $top(i, j) = i - k + 1$  where  $1 \leq k \leq i$  and  $k$  is least possible value such that all cells from  $(k, j)$  to  $(i, j)$  have value 1. Similarly, we can define  $bottom(i, j)$ ,  $left(i, j)$  and  $right(i, j)$  denoting maximum number of consecutive cells on bottom, left and right of  $(i, j)$  respectively.

We can calculate  $top(i, j)$  and  $left(i, j)$  by iterating from the starting of the matrix and updating their values. Refer to the code below to calculate these values.

```
for(int i = 1; i <= R; i++) {
    for(int j = 1; j <= C; j++) {
        if (matrix[i][j] == 0) continue;
        top[i][j] = top[i - 1][j] + 1;
        left[i][j] = left[i][j - 1] + 1;
    }
}
```

}

Similarly, we can calculate  $bottom(i, j)$  and  $right(i, j)$  by iterating from the end of the matrix. After knowing these values, we can calculate number of L-shapes of each type for cell  $(i, j)$ .  $Count(top(i, j), right(i, j))$ ,  $Count(top(i, j), left(i, j))$ ,  $Count(bottom(i, j), right(i, j))$  and  $Count(bottom(i, j), left(i, j))$  denote the number of L-shapes of type 1, 2, 3 and 4 respectively. Thus, for each cell we can add  $Count(top(i, j), left(i, j)) + Count(top(i, j), right(i, j)) + Count(bottom(i, j), left(i, j)) + Count(bottom(i, j), right(i, j))$  to the answer.

$top(i, j)$ ,  $bottom(i, j)$ ,  $left(i, j)$  and  $right(i, j)$  can be calculated in  $O(\mathbf{R} \times \mathbf{C})$  time complexity.  $Count(i, j)$  can be calculate in  $O(\mathbf{R} \times \mathbf{C})$  time complexity. Finally, we need to iterate over each cell of the matrix, and we can update the answer in  $O(1)$  for each cell. Thus, the overall complexity of the solution is  $O(\mathbf{R} \times \mathbf{C})$ .

## Kick Start 2021 - Round A

# Analysis: Rabbit House

Barbara is given a 2D grid with  $R$  rows and  $C$  columns. Her goal is to create a grid such that no two adjacent cells have an absolute difference in height greater than 1. Moreover, she is only allowed to increase the height of cells.

We notice that the cell initially with the largest height ( $H$ ) never requires increasing. Furthermore, Barbara can update its neighbor cells to have a height of  $H - 1$  (unless they already have a height of  $H$ ). Afterwards, she can repeat the process for the cell with the next largest height, until she visits all the cells.

One thing to be careful of in this problem is that the final result can be larger than the limits of a 32-bit integer. Using 64-bit integers avoids WAs due to overflow.

### Test set 1

For this test set, Barbara sees that  $1 \leq R, C \leq 50$ . Therefore, she performs a linear scan over the grid to find the cell with the largest height. Then, she updates the height of its neighbors, and increments the result to account for the increase in height.

Finally, she marks this cell as visited, which can be done via a secondary 2D grid of booleans. She repeats the above process until all cells have been visited, and return the result.

The linear scan can be done in  $O(R \cdot C)$ , and Barbara visits each cell exactly once, so she performs the linear scan  $O(R \cdot C)$  times. The overall time complexity is therefore  $O((R \cdot C)^2)$ . This fits within the limits of the small test set. The additional space complexity is  $O(R \cdot C)$  due to the secondary 2D grid of booleans.

### Test set 2

For this test set,  $1 \leq R, C \leq 300$ . A time complexity of  $O((R \cdot C)^2)$  will not satisfy the time limits.

Barbara still needs to visit the unvisited cell with the largest height in each iteration. However, this can be done in  $O(\log(R \cdot C))$  using a priority queue. In each iteration, she pops the cell with the largest height, updates the height of its neighbors in the priority queue, and increments the result.

The time complexity of the above solution is  $O(R \cdot C \cdot \log(R \cdot C))$ . The additional space complexity is still  $O(R \cdot C)$ , since initially the priority queue contains all the cells.

Note that, depending on the implementation, updating a non-top element in the priority queue might not be an  $O(1)$  operation. In such cases, one trick would be to insert a new element into the queue with the new height, and update the height in the grid. On processing any element, check whether the height in the queue corresponds to the height in the grid, and ignore the element otherwise. This does not change the worst-case time and space complexity, since the maximum number of times a cell can be added to the queue is 4.

### Further Improvements

While not necessary to pass the time limits, the time complexity can be reduced further.

One approach is to replace the priority queue with a list of buckets, each bucket containing a set of cells with a given height. With this approach, Barbara can iterate over the buckets in decreasing order of height in order to visit each cell, then apply the same algorithm as above: update the neighbors (by placing them in their new buckets according to their new height), and remove the visited cells from the list.

By using a hashset for each bucket, insertion and removal becomes an  $O(1)$  operation. She can also use a vector. However, the trick from Test Set 2 is needed to maintain an  $O(1)$  insertion/removal in that case. Iterating over the buckets is linear in the number of buckets, and she will visit at most  $O(\mathbf{R} \cdot \mathbf{C})$  cells.

Let  $G = \max(\mathbf{G}_{i,j})$ . Since no cell has an initial height larger than  $G$ , and she will never increase the height of the cell with the largest height, she observes that the number of buckets is at most  $G$ . This leads to a time complexity of  $O(\mathbf{R} \cdot \mathbf{C} + G)$ .

Barbara can improve this further. Notice that all cells in the final grid will have a height of at least  $G - \mathbf{R} - \mathbf{C} + 2$ . She achieves this value by decreasing the height by 1 with each step away from the highest cell. The maximum number of steps occurs when the highest cell is in a corner of the grid: the opposite corner is  $\mathbf{R} + \mathbf{C} - 2$  steps away.

Now, she can first increase the height of all cells to  $G - \mathbf{R} - \mathbf{C} + 2$ , then apply the same bucketing approach. Except, now the number of buckets is  $\mathbf{R} + \mathbf{C} - 2$ , leading to a time complexity of  $O(\mathbf{R} \cdot \mathbf{C} + \mathbf{R} + \mathbf{C}) = O(\mathbf{R} \cdot \mathbf{C})$ .